

From Flowchart to Java Code

Week 3 — Variables, operators, math functions, type conversion, and reading x to print f(x).

What We'll Write in Java Today

1 Variables & primitive types

int, double, char, boolean, String — the labeled boxes from Week 1, now in code.

2 Operators & the integer-division trap

+ − * / %, precedence, and why 7 / 2 is 3.

3 Math functions

Math.sqrt, Math.pow, Math.abs, Math.round ... and the f(x) pattern.

4 Output: println & printf

Formatting numbers the way you want to see them.

5 Input with Scanner

Read x from the keyboard, compute, print the result.

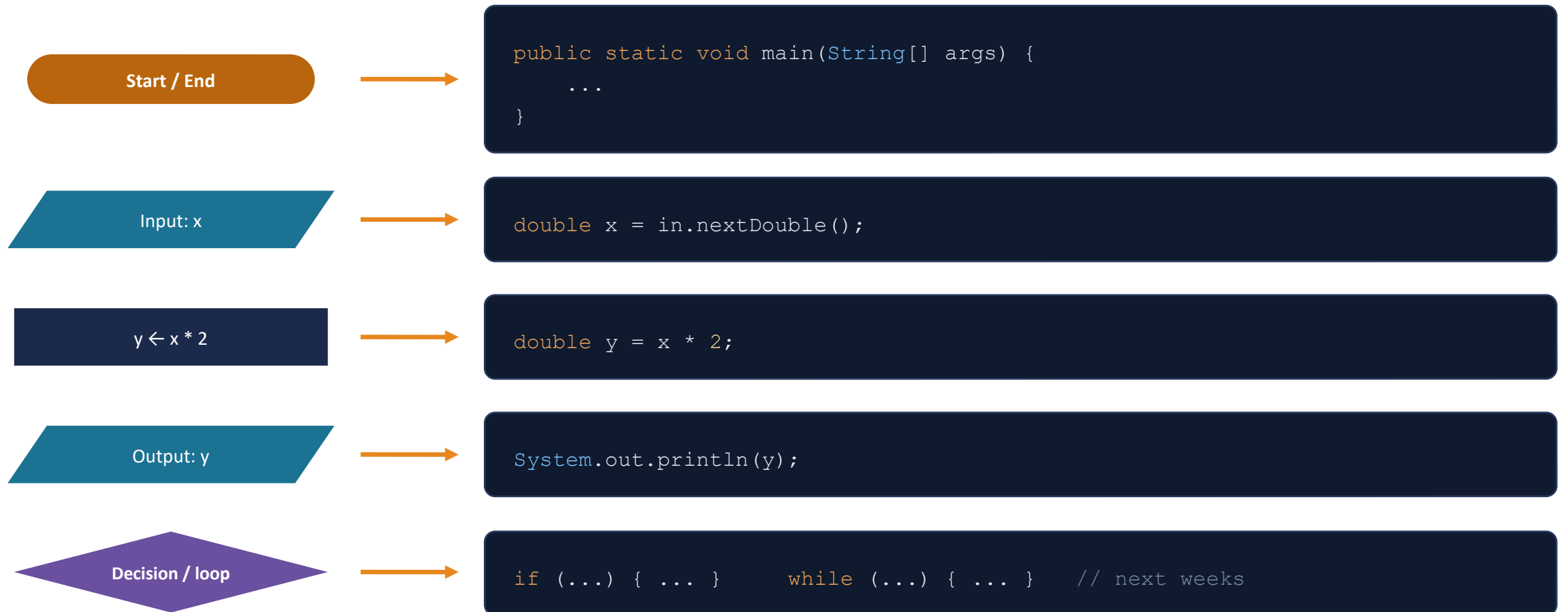
6 Type conversion & casting

When Java converts for you — and when you must ask it to.

HOW TO READ THIS DECK

Every program this week is a flowchart you already know, written in Java. Watch for the same three moves: read, compute, print.

Every Flowchart Symbol Has a Java Counterpart



Anatomy of a Java Program

FirstProgram.java

```
1 public class FirstProgram {  
2     public static void main(String[] args) {  
3         System.out.println("Programming is fun!");  
4     }  
5 }
```

OUTPUT

```
Programming is fun!
```

1

public class FirstProgram

Every program lives in a class. The name must match the file name.

2

{ ... }

Curly braces open and close a block — here, the class body.

3

public static void main(String[] args)

The entry point. Java starts running here — your flowchart's Start.

4

System.out.println(...);

One statement. Every statement ends with a semicolon.

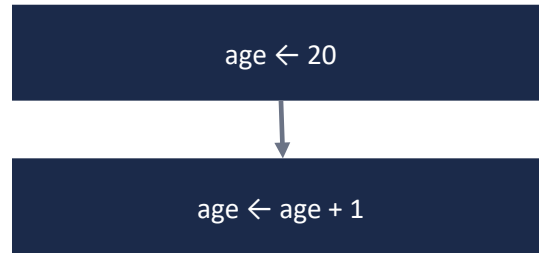
RUN IT

javac FirstProgram.java, then java FirstProgram — or just java FirstProgram.java.

VARIABLES

A Labeled Box for a Value — in Code

WEEK 1



The box named age now holds 21.

JAVA

```
int age = 20;
```

int

type

age

name

=

assign

20

value

```
age = 21;           // reassign: no type this time  
age = age + 1;      // read the old value, store the new one
```

= IS NOT EQUALS

In Java, = means “store the right side into the left side” — the same arrow as ← in a flowchart. Comparing two values uses == (next week).

The Types You'll Use Most

Type	Holds	Example	Size / note
int	whole numbers	42, -7, 0	4 bytes — about ±2.1 billion
long	very large whole numbers	5_000_000_000L	8 bytes — note the L suffix
double	numbers with decimals	3.14, -0.5	8 bytes — about 15 digits of precision
char	one single character	'A', '7', '?'	2 bytes — single quotes
boolean	true or false	true, false	the answer to a yes/no question
String	text (a class, not a primitive)	"Ada", "Hello!"	capital S — double quotes

QUOTES MATTER

'A' is a char (one character, single quotes). "A" is a String (text, double quotes). They are different types and cannot be swapped.

Naming and Declaring Variables

VALID

age

totalPrice

_count

x1

INVALID

2fast starts with a digit

my var contains a space

class reserved keyword

total-price hyphen is an operator



Letters, digits, _ and \$ only; cannot start with a digit.



Case-sensitive: age, Age and AGE are three different variables.



Convention: camelCase, and a name that says what it holds.

Vars.java — inside main

```
int age = 20;
double price = 12.5;
char grade = 'A';
boolean passed = true;
String name = "Ada";
System.out.println(name + " " + age + " " + grade);
System.out.println(price + " " + passed);
```

OUTPUT

```
Ada 20 A
12.5 true
```

Declared, Assigned, Initialized — and final

```
int count;           // declared: a box exists, but it's empty
count = 0;           // assigned
int total = 100;     // declared + initialized in one step
final double TAX = 0.18; // constant: can never change
TAX = 0.20;          // ✗ compile error
```

What if you use a variable before it has a value?

```
int x;
System.out.println(x);
```

COMPILER MESSAGE

```
ErrInit.java:4: error: variable x might not have been initialized
    System.out.println(x);
                   ^
1 error
```

GOOD HABIT

Java refuses to compile if it can't prove a variable has a value. Initialize at declaration, and name constants in UPPER_CASE.

Arithmetic Operators

Op	Meaning	Example	Result
+	addition	7 + 2	9
-	subtraction	7 - 2	5
*	multiplication	7 * 2	14
/	division	7 / 2	3 (both ints → int!)
%	remainder (modulo)	7 % 2	1
+	joins text (String concatenation)	"Java" + "17"	"Java17"

WHY % MATTERS

`n % 10` is the last digit of `n`. `n / 10` drops it. `n % 2` is 0 for even numbers. You'll use these constantly — starting in this week's lab.

The Integer-Division Trap

```
System.out.println(7 / 2);      // 3
System.out.println(7 % 2);      // 1
System.out.println(7 / 2.0);    // 3.5
System.out.println(-7 / 2);     // -3 (cut toward zero)
System.out.println(-7 % 3);     // -1 (sign of the left operand)
```

OUTPUT

```
3
1
3.5
-3
-1
```

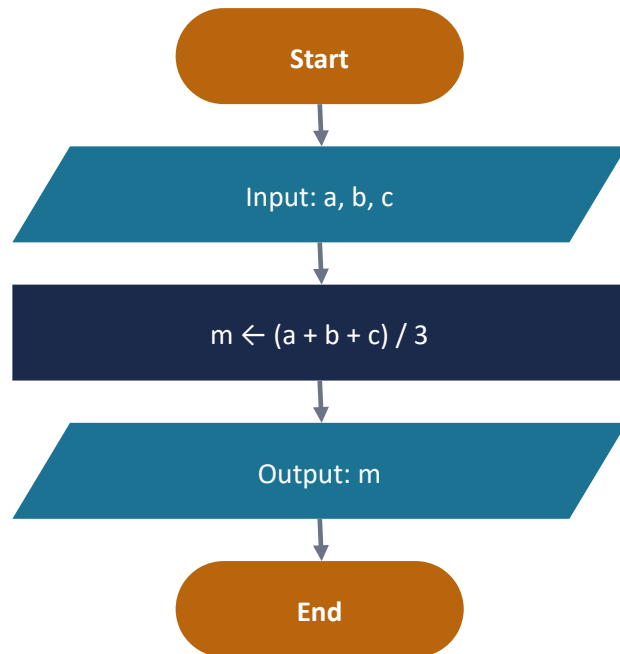
7 / 2 IS 3, NOT 3.5

Java doesn't round — it simply drops the fractional part when both sides are ints. To get 3.5, at least one side must be a double (7 / 2.0).

What decides the result type?

Operands	Result
int ÷ int	int
int ÷ double	double
double ÷ int	double
double ÷ double	double

Average of Three Numbers — and a Trap



AvgTrap.java — inside main

```

int a = 7, b = 8, c = 10;
double wrong = (a + b + c) / 3;      // int / int → 8
double right = (a + b + c) / 3.0;    // int / double → 8.33...
System.out.println(wrong);
System.out.println(right);
    
```

OUTPUT

```

8.0
8.333333333333334
    
```

STORING IN A DOUBLE DOESN'T RESCUE IT

The division already lost its fraction before the result was stored. Fix the operands, not the variable.

Precedence and Shortcut Operators

Who goes first?

1 ()

Ties go left to right.

2 * / %

3 + -

Expression	=	Why
2 + 3 * 4	14	* first
(2 + 3) * 4	20	() first
10 - 4 - 3	3	left to right
7 / 2 * 2	6	3 * 2

Compound.java — inside main

```
int x = 10;
x += 5;    // x = x + 5
x *= 2;    // x = x * 2
x++;       // x = x + 1
x -= 3;    // x = x - 3
System.out.println(x);
```

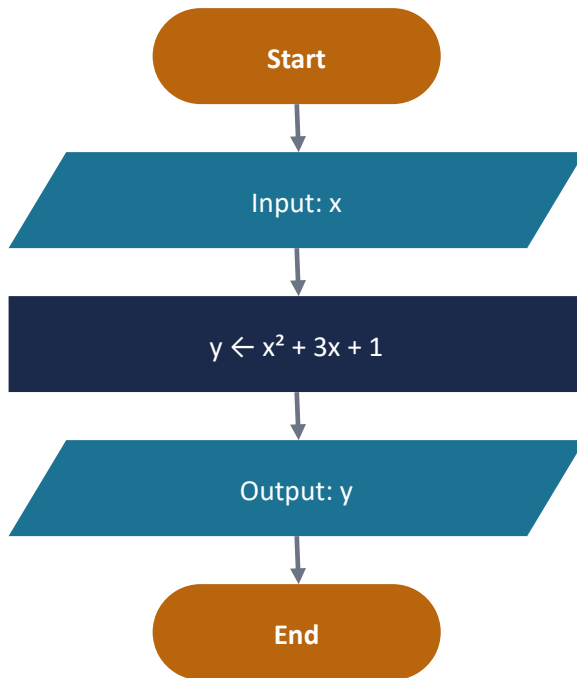
after statement	x
int x = 10;	10
x += 5;	15
x *= 2;	30
x++;	31
x -= 3;	28 ← printed

The Math Class: Functions You Don't Have to Write

Call	What it does	Result
<code>Math.sqrt(16)</code>	square root	4.0
<code>Math.pow(2, 10)</code>	2 raised to the power 10	1024.0
<code>Math.abs(-5)</code>	absolute value	5
<code>Math.max(3, 9)</code>	the larger of two	9
<code>Math.min(3, 9)</code>	the smaller of two	3
<code>Math.round(3.6)</code>	nearest whole number	4
<code>Math.ceil(3.2)</code>	round up	4.0
<code>Math.floor(3.8)</code>	round down	3.0
<code>Math.PI</code>	the constant π	3.141592653589793

sqrt, pow, ceil and floor always return a double — that's why you see 4.0, not 4. round gives a whole number; abs, max and min keep the type you give them.

Read x, Compute f(x), Print the Result



Why 11.0? *Math.pow* returns a double, so the whole expression is a double. $2^2 + 3 \cdot 2 + 1 = 11$.

FxDemo.java

```
import java.util.Scanner;

public class FxDemo {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double x = in.nextDouble();
        double y = Math.pow(x, 2) + 3 * x + 1;
        System.out.println(y);
    }
}
```

TERMINAL

```
$ java FxDemo
2
11.0
```

Hypotenuse of a Right Triangle

$$c = \sqrt{a^2 + b^2}$$

- 1 Read a and b
- 2 Compute c with `Math.sqrt`
- 3 Print c

*`Math.sqrt(a * a + b * b)` and
`Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2))` give
the same answer.*

Hypot.java

```
import java.util.Scanner;

public class Hypot {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double a = in.nextDouble();
        double b = in.nextDouble();
        double c = Math.sqrt(a * a + b * b);
        System.out.println("c = " + c);
    }
}
```

TERMINAL

```
$ java Hypot
3 4
c = 5.0
```

Circle: Area and Perimeter, Two Decimals

$$A = \pi r^2$$

$$P = 2 \pi r$$

Math.PI is a ready-made constant. %.2f in printf rounds the display to two decimals — the variable itself is unchanged.

Circle.java

```
import java.util.Scanner;

public class Circle {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double r = in.nextDouble();
        double area = Math.PI * r * r;
        double perimeter = 2 * Math.PI * r;
        System.out.printf("Area: %.2f\n", area);
        System.out.printf("Perimeter: %.2f\n", perimeter);
    }
}
```

TERMINAL

```
2.5
Area: 19.63
Perimeter: 15.71
```


OUTPUT

print, println, and Escape Sequences

```
System.out.print("Hello, ");  
System.out.println("world");  
System.out.println("Tab\tthere");  
System.out.println("She said \"Hi\"");  
int x = 5;  
System.out.println("x = " + x);
```

OUTPUT

```
Hello, world  
Tab    here  
She said "Hi"  
x = 5
```

PRINT VS PRINTLN

println finishes the line (moves to the next one); print leaves the cursor where it is. + joins text and values: "x = " + x.

Escape sequences

Write	You get
\n	a new line
\t	a tab
\"	a double quote
\\	a backslash

OUTPUT

printf: Formatting What You Print

```
int n = 42;
double x = 3.14159;
String s = "Java";
System.out.printf("%d items\n", n);
System.out.printf("%.2f\n", x);
System.out.printf("%s has %d letters\n", s, s.length());
System.out.printf("%5d|%-5d|%05d\n", n, n, n);
```

OUTPUT

```
42 items
3.14
Java has 4 letters
  42|42   |00042
```

Code	Meaning
%d	an int
%f %.2f	a double (2 decimals)
%s	a String
%c	a char
%n	new line
%5d	width 5, right-aligned
%-5d	width 5, left-aligned
%05d	zero-padded to 5
%%	a literal percent sign

Reading from the Keyboard with Scanner

```
import java.util.Scanner;

Scanner in = new Scanner(System.in);
System.out.print("Name: ");
String name = in.next();
System.out.print("Age: ");
int age = in.nextInt();
System.out.print("GPA: ");
double gpa = in.nextDouble();
System.out.println(name + " is " + age + ", GPA " + gpa);
```

SESSION (typed values: Ada, 20, 3.5)

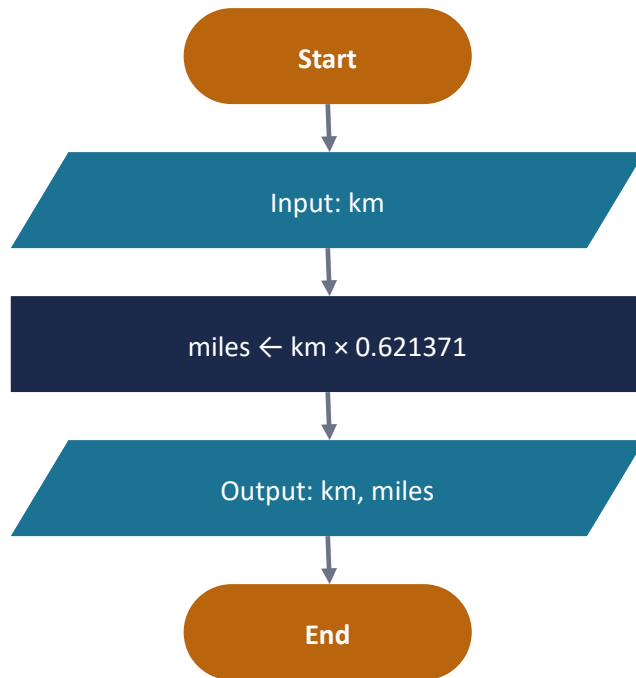
```
Name: Ada
Age: 20
GPA: 3.5
Ada is 20, GPA 3.5
```

Method	Reads	e.g.
<code>nextInt()</code>	an int	20
<code>nextDouble()</code>	a double	3.5
<code>next()</code>	one word	Ada
<code>nextLine()</code>	the whole line	Ada Lovelace
<code>next().charAt(0)</code>	one char	c

THREE LINES TO REMEMBER

import it, create it once with `new Scanner(System.in)`, then call `nextX()` wherever a flowchart says Input.

Flowchart → Java, Line by Line



KmToMiles.java — inside main

```
Scanner in = new Scanner(System.in);
double km = in.nextDouble();
double miles = km * 0.621371;
System.out.printf("%.2f km = %.2f miles\n", km, miles);
```

teal = Input / Output symbol navy = Process symbol

TERMINAL

```
10
10.00 km = 6.21 miles
```

THE PATTERN

Read → compute → print. Nearly every program this semester starts with these three moves.

The nextInt() / nextLine() Trap

BUG

```
int age = in.nextInt();
System.out.print("Name: ");
String name = in.nextLine(); // reads the leftover Enter!
System.out.println "[" + name + "]" + age);
```

TYPED: 20 ↵ (then the program never waits)

```
Age: 20
Name: [] 20
```

FIX

```
int age = in.nextInt();
in.nextLine(); // consume the leftover Enter
System.out.print("Name: ");
String name = in.nextLine();
System.out.println "[" + name + "]" + age);
```

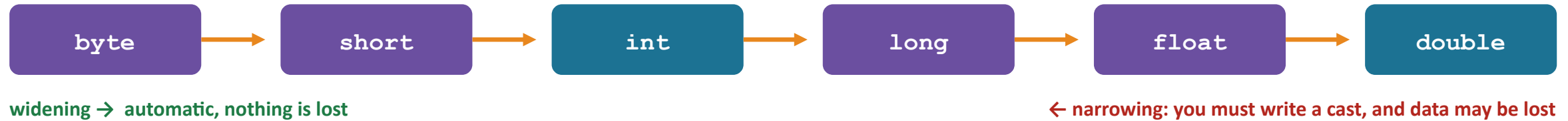
TYPED: 20 ↵ Ada Lovelace ↵

```
Age: 20
Name: Ada Lovelace
[Ada Lovelace] 20
```

WHY IT HAPPENS

`nextInt()` reads the digits but leaves the Enter key in the buffer. The next `nextLine()` finds that empty rest-of-line and returns "" immediately. `next()`, `nextInt()` and `nextDouble()` never have this problem.

Widening Is Free; Narrowing Needs a Cast



```
double d = 7;           // OK: int → double, d is 7.0
int x = (int) 3.99;      // cast: x is 3 (cut off, not rounded)
int y = 3.7;             // ✗ will not compile
```

COMPILER MESSAGE

```
ErrType.java:3: error: incompatible types:
possible lossy conversion from double to int
    int x = 3.7;
```

A CAST TRUNCATES

(int) 3.99 is 3, not 4. If you want the nearest whole number, use `Math.round(3.99)` instead.

Casting Inside Expressions

```
int a = 7, b = 2;
System.out.println((int) 3.99);           // 3
System.out.println(a / b);                 // 3
System.out.println((double) a / b);        // 3.5
System.out.println((double) (a / b));      // 3.0
double d = a;
System.out.println(d);                     // 7.0
```

OUTPUT

```
3
3
3.5
3.0
7.0
```

Where does the cast bite?

Expression	Result
(double) a / b	3.5
(double) (a / b)	3.0
a / (double) b	3.5

A cast applies to what's immediately after it. (double) a / b converts a first, then divides: 7.0 / 2. In (double) (a / b) the integer division happens first — too late.

RULE OF THUMB

Cast the operand, not the finished result. Cast first, then divide.

char ↔ int, and Text ↔ Numbers

Characters are numbers underneath

```
char c = 'A';  
int code = c;           // 65  
char next = (char) (c + 1); // 'B'  
System.out.println('a' - 'A'); // 32
```

OUTPUT (code, next, a-A)

```
65  
B  
32
```

Text to numbers and back

```
int n = Integer.parseInt("42") + 1;  
double d = Double.parseDouble("3.5") * 2;  
String s = String.valueOf(n) + "!";  
String t = "Total: " + 3 + 4;
```

OUTPUT (n, d, s, t)

```
43  
7.0  
43!  
Total: 34
```

"TOTAL: " + 3 + 4

Evaluated left to right: "Total: 3" first, then "Total: 34". Wrap the arithmetic in parentheses — "Total: " + (3 + 4) — to get 7.

When Numbers Don't Fit — or Aren't Exact

```
int big = Integer.MAX_VALUE;  
System.out.println(big);  
System.out.println(big + 1);    // wraps around!  
System.out.println(0.1 + 0.2); // not exactly 0.3
```

OUTPUT

```
2147483647  
-2147483648  
0.30000000000000004
```

CHOOSING A TYPE

Whole numbers beyond ± 2 billion? long. Everyday decimals? double.

int overflow

An int holds at most 2,147,483,647. One more wraps around to the most negative value — silently, with no error.

double precision

Decimals are stored in binary, so some (like 0.1) can't be exact. Tiny errors are normal — never trust == on doubles.

Reading Three Real Compiler Messages

Missing semicolon

```
int x = 5
System.out.println(x);
```

COMPILER MESSAGE

```
ErrSemi.java:3: error: ';' expected
    int x = 5
           ^
```

Lossy conversion

```
int x = 3.7;
```

COMPILER MESSAGE

```
ErrType.java:3: error: incompatible types: possible lossy conversion from double to int
    int x = 3.7;
           ^
```

Uninitialized variable

```
int x;
System.out.println(x);
```

COMPILER MESSAGE

```
ErrInit.java:4: error: variable x might not have been initialized
    System.out.println(x);
                      ^
```

Read it top-down: file and line number → what's wrong → the code line → a caret “^” showing where. Fix the first error first; later ones are often echoes.

PUTTING IT TOGETHER

Everything at Once: A BMI Calculator

1

Read weight (kg) and height (m)

`Scanner + nextDouble`

2

Compute $\text{bmi} = \text{weight} / \text{height}^2$

`operators + parentheses`

3

Print with one decimal

`printf("%.1f")`

PRECEDENCE

weight / height * height is NOT weight / (height * height).
Parentheses matter.

Bmi.java

```
import java.util.Scanner;

public class Bmi {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double weight = in.nextDouble();
        double height = in.nextDouble();
        double bmi = weight / (height * height);
        System.out.printf("BMI: %.1f%n", bmi);
    }
}
```

TERMINAL

```
$ java Bmi
70 1.75
BMI: 22.9
```

Summary & What's Next

- ✓ A variable is a typed, named box: `int age = 20;` — the flowchart's `age` $\leftarrow$ 20.
- ✓ `int / int` drops the fraction; make one operand a double to keep it.
- ✓ `Math.sqrt`, `Math.pow`, `Math.abs`, `Math.round` ... are ready-made functions — `f(x)` is read $\rightarrow$ compute $\rightarrow$ print.
- ✓ `printf` formats output; `Scanner` reads input — and `nextLine()` after `nextInt()` needs care.
- ✓ Widening is automatic; narrowing needs a cast and truncates. Cast the operand, not the result.

Next: if / else and boolean logic — your Week 1–2 decision diamonds, in Java.